

Jaeger Setup

Jaeger Tracing Setup with Dapr

<https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/receiver/> -> link for get all receivers

1. Objective

This setup is used to collect and visualize traces from microservices using Dapr + Jaeger in Kubernetes.

2. Install Jaeger using Helm

Step 1: Add Helm Repository

```
helm repo add jaegertracing https://jaegertracing.github.io/helm-charts
helm repo update
```

Step 2: Install Jaeger

```
helm install jaeger jaegertracing/jaeger \
-n observability \
--create-namespace \
--set provisionDataStore.cassandra=false \
--set allInOne.enabled=false \
--set agent.enabled=false \
--set collector.enabled=true \
--set query.enabled=true \
--set collector.service.otlp.grpc.enabled=true \
--set collector.service.otlp.http.enabled=true
```

Explanation:-

- **collector** → receives traces from services
- **query** → used for UI (to see traces)
- **OTLP enabled** → allows OpenTelemetry-based tracing
- **no Cassandra** → using in-memory storage (good for testing)

3. Configure Dapr Tracing (Using OTLP)

Create a file named:

tracing-config.yaml

apiVersion: dapr.io/v1alpha1

kind: Configuration

metadata:

name: tracing-config

namespace: vizor

spec:

tracing:

samplingRate: "1"

otlp:

endpointAddress: "http://jaeger-collector.observability.svc.cluster.local:4317"

isSecure: false

4. Apply Configuration

kubectl apply -f tracing-config.yaml

5. Update All Services

In every service deployment YAML, add:

metadata:

annotations:

dapr.io/enabled: "true"

dapr.io/app-id: "your-app-name"

dapr.io/app-port: "your-port"

dapr.io/config: "tracing-config" # important

6. Redeploy Application

helm upgrade --install vizor ./ -n vizor

7. Access Jaeger UI

Port Forward

kubectrl port-forward svc/jaeger-query -n observability 16686:16686

Open in Browser

<http://localhost:16686>

vizor services flow Performance

Distributed Traces Vizor Performance

Service Flow:

api-proxy → callocal/core-service/user/userlogin
Role: Client (api-proxy calling core-service via Dapr)
Status Code: 200
Duration: 1.22s (Average)

Service Flow:

api-proxy → callocal/engagement-service/home/getroundtableactivities
Role: Client (api-proxy calling engagement-service via Dapr)
Status Code: 200
Duration: 24.84s (Very Slow)

Service Flow:

api-proxy → callocal/core-service/client/ddl/0
Role: Client (api-proxy calling core-service via Dapr)
Status Code: 200
Duration: 462.74ms (Good)

Service Flow:

api-proxy → callocal/engagement-service/roundtable
Role: Client (api-proxy calling engagement-service via Dapr)
Status Code: 200
Duration: 497.07ms (Good)

Service Flow:

api-proxy → callocal/core-service/user/ar/directors
Role: Client (api-proxy calling core-service via Dapr)
Status Code: 200
Duration: 1.25s (Average)

Service Flow:

api-proxy → callocal/core-service/client
Role: Client (api-proxy calling core-service via Dapr)

Status Code: 200
Duration: 2.54s (Slow)

Service Flow:

api-proxy → callocal/core-service/user/ar/associates
Role: Client (api-proxy calling core-service via Dapr)
Status Code: 200
Duration: 1.25s (Average)

Service Flow:

api-proxy → callocal/engagement-service/home/gethomepageallsections
Role: Client (api-proxy calling engagement-service via Dapr)
Status Code: 200
Duration: 25.83s (Very Slow)

Service Flow:

api-proxy → callocal/core-service/user
Role: Client (api-proxy calling core-service via Dapr)
Status Code: 200
Duration: 9.15s (Very Slow)

Service Flow:

api-proxy → callocal/core-service/user/startmimic
Role: Client (api-proxy calling core-service via Dapr)
Status Code: 200
Duration: 1.46s (Average)

Service Flow:

api-proxy → callocal/core-service/user/logout
Role: Client (api-proxy calling core-service via Dapr)
Status Code: 200
Duration: 677.01ms (Good)

Service Flow:

core-service → calllocal/engagement-service/common/token-revoked

Role: Client (core-service calling engagement-service via Dapr)

Status Code: 200

Duration: 189.89ms (Excellent)

Service Flow:

core-service → calllocal/engagement-service/refreshlog/getrefreshlog

Status Code: 200

Duration: 2.09s (slow)

Service Flow:

core-service → calllocal/engagement-service/survey

Status Code: **Code(500): Internal Server Error**

Duration: 1.67s (Average)

Service Flow:

core-service → calllocal/engagement-service/survey/getsurveyemail

Status Code: 200

Duration: 59.73ms (Excellent)

Service Flow:

core-service → calllocal/engagement-service/survey/section/question

Status Code: 200

Duration: **5.51s (Slow)**

Service Flow:

core-service → calllocal/engagement-service/survey/launch/survey

Status Code: 200

Duration: **4.65s (Slow)**

Service Flow:

core-service → calllocal/engagement-service/meetingagenda

Status Code: 200

Duration: 3.54s (Slow)

Service Flow:

core-service → calllocal/engagement-service/meetingreport

Status Code: 200

Duration: 1.63s (Average)

Service Flow:

core-service → calllocal/engagement-service/meetingcalender/calender

Status Code: 200

Duration: 13.36ms (excellent)

Service Flow:

core-service → calllocal/engagement-service/meetingcalender/create

Status Code: 200

Duration: 2.72s (Slow)

Service Flow:

core-service → calllocal/engagement-service/meetingcalenderexport/csv

Status Code: 200

Duration: 782.16ms (good)

Service Flow:

core-service → calllocal/engagement-service/meetingsummit/1/prospect

Status Code: 200

Duration: 2.62s (Slow)

Service Flow:

core-service → calllocal/engagement-service/meetingsummit/add-prospects

Status Code: Code(500): Internal Server Error

Duration: 8.52s (Very Slow)

Service Flow:

core-service → calllocal/engagement-service/meetingsummit/test-email

Status Code: 200

Duration: 691.21ms (good)

Service Flow:

core-service → calllocal/interaction-service/community

Status Code: 200

Duration: 3.69s (slow)

Service Flow:

core-service → calllocal/core-service/content/createcontent

Status Code: 200

Duration: 36.2ms (excellent)

Service Flow:

core-service → calllocal/core-service/content/createcontent

Status Code: 200

Duration: 3.74s (slow)

openobserve for logs

Openobserve setup for logs

1. Created namespace for OpenObserve

```
kubectl create namespace openobserve
```

2. Deployed OpenObserve

Via helm :-

```
helm repo add openobserve https://openobserve.github.io/helm-charts
helm repo update
```

```
kubectl create namespace openobserve
helm install openobserve openobserve/openobserve-standalone \
  -n openobserve \
```

For update:-

```
helm upgrade openobserve openobserve/openobserve-standalone \
  -n openobserve \
  -f values.yaml
```

For see values.yaml:-

```
helm show values openobserve/openobserve-standalone
```

```
helm show values openobserve/openobserve-standalone > values.yaml
```

Via kubectl :-

```
kubectl run openobserve \
  --image=public.ecr.aws/zinclabs/openobserve:latest \
  --port=5080 \
  -n openobserve \
  --env="ZO_ROOT_USER_EMAIL=admin@example.com" \
  --env="ZO_ROOT_USER_PASSWORD=Admin@123"
```

Expose service

```
kubectl expose pod openobserve \
  --type=NodePort \
  --port=5080 \
  -n openobserve
```

3. Verified deployment

```
kubectl get pods -n openobserve
kubectl get svc -n openobserve
```

👉 Ensured:

- Pod is **Running**
 - Service exposed on **port 5080**
-

4. Accessed OpenObserve UI

<http://localhost:5080>

👉 Logged in using custom credentials:

[admin@example.com](#) / Admin@123

5. Created logging namespace

```
kubectl create namespace logging
```

👉 Used for log collector (Fluent Bit)

6. Installed Fluent Bit using Helm

```
helm repo add fluent https://fluent.github.io/helm-charts
helm repo update
helm install fluent-bit fluent/fluent-bit -n logging
```

For upgrade :-

```
helm upgrade fluent-bit fluent/fluent-bit \
-n logging \
-f fluent-values.yaml
```

👉 Fluent Bit acts as:

Log Collector Agent

7. Modified Fluent Bit configuration

```
kubectrl edit configmap fluent-bit -n logging
```

Removed default Elasticsearch outputs

```
[OUTPUT]
  Name es
  ...
```

Added OpenObserve output

```
[OUTPUT]
  Name http
  Match *
  Host openobserve.openobserve
  Port 5080
  URI /api/default/default/_json
  Format json
  json_date_key timestamp
  json_date_format iso8601
  HTTP_User admin@example.com
  HTTP_Passwd Admin@123
```

👉 This step is the **most important**

8. Restarted Fluent Bit

`kubectl rollout restart daemonset fluent-bit -n logging`

👉 Applies new config

9. Verified log ingestion

`kubectl logs -n logging -l app.kubernetes.io/name=fluent-bit`

Saw:

HTTP status=200
successful: X, failed: 0

👉 Means:
Logs successfully sent to OpenObserve

10. Viewed logs in OpenObserve UI

Steps:

1. Go to **Logs** → **Streams**
 2. Select stream → **default**
 3. Set time range → Last 15 min
 4. Run query
-

Example queries

All logs:

```
select * from "default" limit 100
```

Your app logs:

```
SELECT * FROM "default"  
WHERE kubernetes_namespace_name = 'vizzor'
```

LIMIT 100

Specific service:

```
SELECT * FROM "default"  
WHERE kubernetes_pod_name LIKE '%core%'  
LIMIT 100
```

log: [05:55:40 DBG] Execution loop DelayedJobScheduler:91eb968c stopped in 0 ms,

Only for log:-

```
SELECT log FROM "default"  
WHERE kubernetes_labels_vizor = 'interaction-service'
```

openobserve for traces

1. cert-manager (TLS manager)

cert-manager

Purpose:

- Manages TLS certificates in Kubernetes
- Required for secure webhook / mTLS / operators

Installed with:

kubectl apply -f

<https://github.com/cert-manager/cert-manager/releases/latest/download/cert-manager.crds.yaml>

helm repo add jetstack https://charts.jetstack.io

helm repo update

helm install cert-manager jetstack/cert-manager \

--namespace cert-manager \

--version v1.14.5 \

--set installCRDs=false

2. OpenTelemetry Operator

OpenTelemetry Operator

Purpose:

- Manages OpenTelemetryCollector CRDs
- Deploys and manages collectors automatically

Installed with:

helm repo add open-telemetry https://open-telemetry.github.io/opentelemetry-helm-charts

helm repo update

helm install otel-operator open-telemetry/opentelemetry-operator \

-n observability --create-namespace

3. OpenTelemetry Collector

Purpose:

- Receives traces from Dapr
 - Sends to OpenObserve
-

3. OTEL COLLECTOR YAML

apiVersion: opentelemetry.io/v1beta1

kind: OpenTelemetryCollector

metadata:

name: otel

namespace: observability

spec:

mode: deployment

config:

receivers:

otlp:

protocols:

grpc:

endpoint: 0.0.0.0:4317

http:

endpoint: 0.0.0.0:4318

processors:

batch: {}

exporters:

otlphttp/openobserve:

endpoint: http://openobserve.openobserve:5080/api/default

headers:

Authorization: Basic YWRtaW5AZXhhbXBsZS5jb206RHVDSml4dmRocXZuQ3QyTw==

stream-name: default

service:

pipelines:

traces:

receivers: [otlp]

processors: [batch]

```
exporters: [otlphttp/openobserve]
```

Apply collector:

```
kubectl apply -f otel-collector.yaml
```

4. Dapr Tracing Configuration (IMPORTANT ADDITION)

```
apiVersion: dapr.io/v1alpha1
```

```
kind: Configuration
```

```
metadata:
```

```
  name: tracing-config
```

```
  namespace: vizor
```

```
spec:
```

```
  tracing:
```

```
    samplingRate: "1"
```

```
    otel:
```

```
      endpointAddress: "otel-collector.observability:4317"
```

```
      isSecure: false
```

```
      protocol: grpc
```

5. DATA FLOW:-

[Dapr sidecars in microservices]

↓ (OTLP gRPC 4317)

[OpenTelemetry Collector]

↓

(process + batch)

↓

[OpenObserve ingestion API]

↓

Traces / Logs / Metrics Storage

↓

OpenObserve UI (Trace Explorer)

openobserve for metrics

1. Add Helm Repository

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

2. Install Prometheus

We install Prometheus in the `observability` namespace:

```
helm install prometheus prometheus-community/prometheus \
-n observability --create-namespace
```

3. Verify Prometheus is Running

```
kubectl get pods -n observability
kubectl get svc -n observability
```

You should see:

- prometheus-server
 - alertmanager
 - node-exporter (if enabled)
-

4. Edit Prometheus ConfigMap (IMPORTANT STEP)

Now we connect Prometheus → OpenObserve.

Edit ConfigMap:

```
kubectl edit configmap prometheus-server -n observability
```

5. Add remote_write to OpenObserve

Inside `prometheus.yml`, add:

```
remote_write:
- url: http://openobserve.openobserve:5080/api/default/prometheus/api/v1/write
  queue_config:
    max_samples_per_send: 10000
  basic_auth:
    username: admin@example.com
    password: DuCJb8vdhqvnCt2O
  headers:
    stream-name: default
```

6. Restart Prometheus

```
kubectl rollout restart deployment prometheus-server -n observability
```

7. Check Logs (Verify Data Flow)

```
kubectl logs -n observability deploy/prometheus-server -f
```

Data Flow :-

[Kubernetes Cluster]



Prometheus Server
(scrapes metrics from)

- nodes
- pods
- services



remote_write



OpenObserve Ingestion API



Metrics Stream (default)



OpenObserve UI (Dashboards / Queries)

openobserve-collector setup via
podman

Openobserve setup for traces logs metrics with openobserve collector and run collector on k8s and openobserve ui on podman

1. Vizor Services Send Traces Through Dapr

The services with Dapr tracing enabled are:

- `vizor-core-service`
- `vizor-engagement-service`
- `vizor-interaction-service`
- `api-proxy`

Their Kubernetes deployments contain annotations like:

```
dapr.io/enabled: "true"  
dapr.io/config: "tracing-config"
```

2. Dapr Tracing Configuration

Your Dapr tracing configuration points to the OpenObserve Collector Gateway.

Example:

```
apiVersion: dapr.io/v1alpha1  
kind: Configuration  
metadata:  
  name: tracing-config  
  namespace: vizor
```

```
spec:  
  tracing:  
    samplingRate: "1"
```

```
otel:
```

```
endpointAddress:
o2c-openobserve-collector-gateway-collector.openobserve-collector.svc.cluster.local:4317
isSecure: false
protocol: grpc
```

3. Trace Flow

The actual trace flow is:

```
Vizor Service
↓
Dapr Sidecar
↓
OpenObserve Collector Gateway (4317)
↓
OpenObserve Collector
↓
OpenObserve UI/Storage
```

4. OpenObserve Collector Running in Kubernetes

Important Helm command for install openobserve collector:- from data sources od openobserve ui :-

```
helm --namespace openobserve-collector upgrade --install o2c
openobserve/openobserve-collector \
--reuse-values \
--set k8sCluster=nuage-cluster \
--set exporters.otlphttp/openobserve.endpoint=http://169.254.1.2:5080/api/default/ \
--set exporters.otlphttp/openobserve.headers.Authorization='Basic
<OPENOBSERVE_BASIC_AUTH>' \
--set exporters.otlphttp/openobserve_k8s_events.endpoint=http://169.254.1.2:5080/api/default/
\
--set exporters.otlphttp/openobserve_k8s_events.headers.Authorization='Basic
<OPENOBSERVE_BASIC_AUTH>'
```

The OpenObserve Collector is installed in Kubernetes using Helm.

5. Collector Gateway Service

The collector gateway service is:

o2c-openobserve-collector-gateway-collector.openobserve-collector.svc.cluster.local

Ports:

4317 -> OTLP gRPC

4318 -> OTLP HTTP

Use:

- 4317 for OTLP gRPC
- 4318 for OTLP HTTP

Your Dapr setup uses:

4317 (gRPC)

6. OpenObserve Running Through Podman

```
podman run -d \  
  --name openobserve \  
  -p 5080:5080 \  
  -v ~/openobserve-data:/data \  
  -e ZO_ROOT_USER_EMAIL="root@example.com" \  
  -e ZO_ROOT_USER_PASSWORD="OpenObserve123" \  
  public.ecr.aws/zinclabs/openobserve:latest
```

Currently, OpenObserve itself is NOT running inside Kubernetes.

That means the collector exports data to an external OpenObserve instance running through Podman on the host machine.

Current endpoint for collector :- <http://169.254.1.2:5080/api/default>

Command for check correct endpoint :-

```
kubectrl run oo-test --rm -i --restart=Never --image=curlimages/curl -- \  
curl -I http://169.254.1.2:5080/
```

9. Important Endpoint Rule

During collector installation:

Use the endpoint reachable from Kubernetes.

Do NOT use:

- Browser URL
- localhost
- 127.0.0.1

Because Kubernetes pods cannot access your host localhost directly.

For your current setup, use:

`http://169.254.1.2:5080/api/default/`

10. How to Check the Collector Endpoint Later

Check live Helm values

```
helm get values o2c -n openobserve-collector
```

Flow :-

Vizor Services

↓

Dapr Sidecars

↓

OpenObserve Collector Gateway (4317)

↓

OpenObserve Collector

↓

Exports telemetry to:

`http://169.254.1.2:5080/api/default/`

↓

OpenObserve running in Podman

headlamp and openobserve local store
data

```
helm repo add headlamp https://kubernetes-sigs.github.io/headlamp/  
helm repo update  
helm install my-headlamp headlamp/headlamp --namespace headlamp
```

```
4.kubectl create token my-headlamp --namespace kube-system
```

```
#####  
#
```

Where openobserve data actually stored in local how can we get that:-

Your local machine (inside Podman container storage)

Commands to get that data like logs, traces and metrics:-

STEP 1 — Check OpenObserve Pod

```
kubectl get pods -n openobserve
```

STEP 2 — Check mounted storage

```
kubectl describe pod openobserve-openobserve-standalone-0 -n openobserve
```

Look for:

Mounts:

/data from data (rw)

Meaning:

OpenObserve stores data in **/data**.

STEP 3 — Check PVC

```
kubectl get pvc -n openobserve
```

Output:

data-openobserve-openobserve-standalone-0

STEP 4 — Check PV details

kubectl describe pv pvc-204ff9d6-e647-4d1f-9e5a-e1f7aca39c11

Important part:

Path:

/opt/local-path-provisioner/pvc-204ff9d6...

and:

selected-node: nuage-cluster-worker

Meaning:

actual data exists inside worker node container.

STEP 5 — Verify KIND uses Podman

kind get clusters

You saw:

using podman due to KIND_EXPERIMENTAL_PROVIDER

STEP 6 — Enter KIND worker node container

podman exec -it nuage-cluster-worker sh

Now you are INSIDE Kubernetes worker node filesystem.

STEP 7 — Go to actual PV storage

```
cd /opt/local-path-provisioner/
```

STEP 8 — List all PVC storage directories

```
ls
```

You saw:

```
pvc-204ff9d6-e647-4d1f-9e5a-e1f7aca39c11_openobserve_data-openobserve-o  
penobserve-standalone-0
```

STEP 9 — Enter OpenObserve storage

```
cd
```

```
pvc-204ff9d6-e647-4d1f-9e5a-e1f7aca39c11_openobserve_data-openobserve-o  
penobserve-standalone-0
```

STEP 10 — See actual OpenObserve storage folders

```
ls
```

You saw:

```
cache db mmdb stream tmp wal
```

STEP 11 — Go to trace storage

```
cd stream/files/default/traces/default
```

STEP 12 — Navigate by date

cd 2026/05/07/06

STEP 13 — View actual stored trace files

ls

You saw:

74580506887239761927564.parquet

need to explore

1.Retention in openobserve

Option	What it is	Good for	Pros	Cons
Local Disk (PVC)	Data stored inside Kubernetes node/pod		Dev / testing	Easy, no setup Data lost if pod/node dies
MinIO	Self-hosted S3-compatible storage		Private cloud / lab	Free, S3-like, controllable You must manage it
AWS S3	Cloud object storage	Production (best)	Highly durable, scalable, cheap long-term	Paid, internet required
Data Lake (Iceberg/BigQuery)		Analytics storage layer	Big data analytics	Powerful analytics Complex setup

Query for read logs

#find the reason for 500 via exact timestamp:-

```
SELECT _timestamp, time, k8s_deployment_name, k8s_pod_name, body
FROM default
WHERE k8s_namespace_name = 'vizer-prod'
AND k8s_deployment_name = 'vizer-engagement-service'
AND _timestamp <= 1780643914322712
AND (
    body LIKE '%SQL Exception occurred%'
    OR body LIKE '%SqlException%'
)
ORDER BY _timestamp DESC
LIMIT 5
```

#find reason for 500:-

```
SELECT *
FROM default
WHERE k8s_deployment_name = 'vizer-engagement-service'
AND _timestamp BETWEEN 1780665751964416 - 300000000
AND 1780665751964416 + 300000000
AND (
    body LIKE '%Exception%'
    OR body LIKE '%Error%'
    OR body LIKE '%SqlException%'
    OR body LIKE '%Timeout%'
    OR body LIKE '%failed%'
    OR body LIKE '%null%'
)
ORDER BY _timestamp DESC
```

#Query for got the count of username:-

```
SELECT
    str_match(body, 'username=("[^"]+)"') AS username,
    COUNT(*) AS username_count,
    max(_timestamp) AS latest_time
```

```

FROM default
WHERE k8s_deployment_name = 'keycloak'
AND body LIKE '%custom_required_action="UPDATE_PASSWORD"%'
AND body LIKE '%response_type="code"%'
AND body LIKE '%remember_me="false"%'
AND body LIKE '%response_mode="fragment"%'
AND body LIKE '%username="% '
GROUP BY username
ORDER BY latest_time DESC

```

#Query for see username with body in keycloak:-

```

SELECT *
FROM default
WHERE k8s_deployment_name = 'keycloak'
AND body LIKE '%custom_required_action="UPDATE_PASSWORD"%'
AND body LIKE '%response_type="code"%'
AND body LIKE '%remember_me="false"%'
AND body LIKE '%response_mode="fragment"%'
AND body LIKE '%username="% '
ORDER BY _timestamp DESC

```

#Query for see only username in keycloak:-

```

SELECT
  regexp_match(body, 'username="([^\"]+)"') AS username,
  max(_timestamp) AS latest_time
FROM default
WHERE k8s_deployment_name = 'keycloak'
AND body LIKE '%custom_required_action="UPDATE_PASSWORD"%'
AND body LIKE '%response_type="code"%'
AND body LIKE '%remember_me="false"%'
AND body LIKE '%response_mode="fragment"%'
AND body LIKE '%username="% '
GROUP BY username
ORDER BY latest_time DESC

```

#Query for see error related 500:-

```
SELECT *
FROM default
WHERE (k8s_namespace_name = 'vizzor-prod' OR k8s_namespace_name =
'traefik')
AND downstreamstatus = 500
```

#Query for see traces with status code.

```
service_name IN (
  'keycloak',
  'traefik',
  'api-proxy',
  'core-service',
  'engagement-service',
  'interaction-service'
)
AND http_response_status_code = 500
```

#For logs :-

Check for failed api request:-

```
k8s_namespace_name = 'vizzor-prod'
AND k8s_deployment_name = 'vizzor-engagement-service'
AND body NOT LIKE '%/health%'
AND (
  body LIKE '% - 400 %'
  OR body LIKE '% - 401 %'
  OR body LIKE '% - 403 %'
  OR body LIKE '% - 404 %'
  OR body LIKE '% - 500 %'
  OR body LIKE '% - 502 %'
  OR body LIKE '% - 503 %'
```

```
OR body LIKE '% - 504 %'
)
```

Check for successfully api request:-

```
k8s_namespace_name = 'vizor-prod'
AND k8s_deployment_name = 'vizor-engagement-service'
AND body NOT LIKE '%/health%'
AND (
  body LIKE '% - 200 %'
  OR body LIKE '% - 201 %'
  OR body LIKE '% - 204 %'
)
```

#Check database related logs:-

```
k8s_namespace_name = 'vizor-prod'
AND k8s_deployment_name = 'vizor-engagement-service'
AND (
  body LIKE '%Executed DbCommand%'
  OR body LIKE '%Failed executing DbCommand%'
  OR body LIKE '%SqlException%'
)
```

#Only error related logs :- check all

```
k8s_namespace_name = 'vizor-prod'
AND body NOT LIKE '%/health%'
AND body NOT LIKE '%Connection id%'
AND body NOT LIKE '%Request starting%'
AND (
  body LIKE '% - 4%'
  OR body LIKE '% - 5%'
  OR body LIKE '%ERR%'
  OR body LIKE 'fail:%'
  OR body LIKE '%Exception:%'
  OR body LIKE '%SqlException%'
)
```

```
{
  _timestamp:2026-05-19 06:11:35.392,
  body:      Request finished HTTP/1.1 POST
http://api-proxy-874cc5c9d-8q9cf/api/Reports/ReportServer/Reserved.ReportViewerWebCont
rol.axd?OpType=SessionKeepAlive&ControlID=936fcfbfa43cf48dd884244a66c2e2974 - 500 -
}
```

```
{
  _timestamp: 2026-05-19 20:39:31.785
  body: [15:09:31 INF] Request finished HTTP/1.1 POST
  http://127.0.0.1:8080/Home/saveModifySurveyResponse - 500 null application/json
}
```

```
{
  _timestamp: 2026-05-20 03:06:37.449,
  body: [21:36:37 INF] Request finished HTTP/1.1 POST
  http://127.0.0.1:8080/Survey/saveParticipants - 500 null application/json 55.0395
}
```

[illegible]

```
body:      Request finished HTTP/1.1 GET
http://api-proxy-874cc5c9d-sb9kq/User/getUserById - 401 -
application/json;charset=utf-8 5.1658ms,
```

What We Check	Why
Errors	Find failures
HTTP status	Success/failure
API endpoints	Which API called
Response time	Performance
Exceptions	Root cause
Database logs	SQL problems
Auth logs	Token/login issues
Kubernetes info	Which pod/node failed
Timestamps	When issue happened
Trace IDs	Request tracking
Warnings	Early problems
Traffic patterns	System usage

Vrl function for convert body (strings to json):-

```

if is_string(.body) {
    parsed_body, err = parse_json(.body)

    if err == null {
        .body = parsed_body
    }
}

```

Different vrl function what i used in my setup :-

```

.pipeline_test = "pipeline_is_working"

if is_string(.body) {
    cleaned = string!(.body)
}

```

```

cleaned = replace!(cleaned, r'\x1b\[0-9;]*m', "")

json_match, err = parse_regex(cleaned, r'(?P<json>\{.*\})$')

if err == null {
    parsed, err2 = parse_json(json_match.json)

    if err2 == null {
        .body_json = parsed
        .body = parsed
        .body_text = cleaned
    }
}
}

```

Vrl function for error :-

```

.error_pipeline = "working"

if is_string(.body) {
    body_text = string!(.body)
    body_text = replace!(body_text, r'\x1b\[0-9;]*m', "")

    .body_text = body_text

    if match(body_text, r'(?i)\b(ERR|ERROR|Exception|Failed|Failure|Fatal)\b') {
        .is_error = true
        .level = "error"

        if match(body_text, r'(?i)Failed executing DbCommand') {
            .error_type = "database_error"
        } else if match(body_text, r'(?i)Unauthorized|401') {
            .error_type = "unauthorized_error"
        } else if match(body_text, r'(?i)timeout|timed out') {
            .error_type = "timeout_error"
        } else if match(body_text, r'(?i)Exception') {
            .error_type = "exception_error"
        } else if match(body_text, r'(?i)500|Internal Server Error') {
            .error_type = "server_error"
        } else {
            .error_type = "general_error"
        }

        .error_message = body_text
    } else {
        .is_error = false
    }
}
}

```

VRL Function for any security related issue

```
msg = downcase(string!(.body.message ?? .body ?? ""))
```

```
.failed_login = false  
.sql_injection_attempt = false  
.bruteforce_possible = false
```

```
if contains(msg, "failed login") ||  
  contains(msg, "invalid password") ||  
  contains(msg, "authentication failed") ||  
  contains(msg, "login failed") {  
  .failed_login = true  
}
```

```
if contains(msg, "' or 1=1") ||  
  contains(msg, "union select") ||  
  contains(msg, "drop table") ||  
  contains(msg, "information_schema") ||  
  contains(msg, "--") ||  
  contains(msg, "sleep(") {  
  .sql_injection_attempt = true  
}
```

```
if .failed_login == true {  
  .bruteforce_possible = true  
}
```

Latest vrl function thats use inside vizormonitoring

VRL stands for **Vector Remap Language**. It's the language used in OpenObserve pipelines to transform logs before they are stored.

Your script:

Use this cleaner VRL:

```
.pipeline_test = "pipeline_is_working"
```

```
if is_string(.body) {
```

```
  cleaned = string!(.body)
```

```
  cleaned = replace!(cleaned, r'\x1b\[0-9;]*m', "")
```

```
  json_match, err = parse_regex(cleaned, r'(?P<json>\{.*\})$')
```

```

if err == null {

    parsed, err2 = parse_json(json_match.json)

    if err2 == null {

        . = merge!(., parsed)

        del(.body)

        del(.body_json)

        del(.body_text)

    }

}

}

```

is doing several operations:

1. Adds a field

```
.pipeline_test = "pipeline_is_working"
```

2. Checks if **body** is a string

```
if is_string(.body)
```

3. Removes ANSI color codes

```
replace!(...)
```

4. Extracts JSON using regex

```
parse_regex(...)
```

5. Converts JSON string into an object

```
parse_json(...)
```

6. Splits JSON into separate fields

```
. = merge!(., parsed)
```

```
request_referer:
https://vizzor.roundtables.us/vizzor/survey/manage-report-builder/publish-preview,
request_x_original_url: /v1.0/invoke/core-service/method/api/Reports/render/verify,

url_full:
http://10.233.83.58:80/v1.0/invoke/core-service/method/api/Reports/render/verify,

request_x_original_url:
/v1.0/invoke/engagement-service/method/Survey/saveParticipants,

request_x_original_url:
/v1.0/invoke/core-service/method/api/Reports/ReportServer/Reserved.ReportViewerWebCont
rol.axd?OpType=SessionKeepAlive&ControlID=1a46cb61812842539e4b9081d5b354f8,
```

openobserve alert

Setup for alert in openobserve

Step 1: Verify logs exist

Run:

```
SELECT *  
FROM default  
WHERE downstreamstatus = 500
```

If you see records, your field is correct.

Step 2: Create Alert

Go to:

Alerts → Create Alert

Choose:

Stream Type: Logs
Stream: default
Alert Type: Real-time

Step 3: Configure the condition

Instead of returning all columns, use a count query:

```
SELECT count(*) as error_count  
FROM default  
WHERE downstreamstatus = 500
```

Set the alert trigger condition:

`error_count > 0`

Meaning:

If at least one 500 error log arrives → trigger alert

Step 4: Configure notification

Create a destination:

Alerts → Destinations

Choose one:

- Email
- Slack
- Microsoft Teams
- Webhook

For learning, Email is easiest.

Step 5: Save

Example:

Name: HTTP 500 Error Alert

Severity: Critical

Enrichment table in pipeline:-

Why use Enrichment Tables?

1. IP Address → Location

Input log:

```
{  
  "client_ip": "10.0.0.5"  
}
```

Enrichment table:

client_ip	location
10.0.0.5	Mumbai
10.0.0.6	Delhi

Enrichment table in pipeline for status code :-

status_code,description,severity
200,Success,Info
301,Redirect,Info
400,Bad Request,Warning
401,Unauthorized,Warning
403,Forbidden,Warning
404,Not Found,Warning
500,Internal Server Error,Critical
502,Bad Gateway,Critical
503,Service Unavailable,Critical
504,Gateway Timeout,Critical

Enrichment table in pipeline for location :-

client_ip,location,environment
10.233.95.128,Kubernetes Cluster,Production
10.233.95.129,Kubernetes Cluster,Production
192.168.1.10,Office Network,Development
192.168.1.20,Office Network,Testing

Official semantic convention means OpenTelemetry standard name.

Example: `service.name`, `k8s.pod.name`, `http.request.method`.

In OpenObserve these often become underscore style:

`service.name` -> `service_name`
`k8s.pod.name` -> `k8s_pod_name`
`http.request.method` -> `http_request_method`

OpenTelemetry uses semantic conventions to give common standard names for traces, metrics, logs, and resources.

What is custom/vendor/body-flattened?

1. Custom fields

These are fields created by your app, log format, or pipeline.

Example from your logs:

body_json_request_path
body_json_status_code
body_json_message
body_json_traceid
body_duration_ms

These are **not OpenTelemetry official names**. They come from your application logs or OpenObserve parsing.

Kubernetes-Security

1. Kubernetes Security Fundamentals

Kubernetes security means **protecting your Kubernetes cluster, applications, containers, network, secrets, and data** from attackers, unauthorized users, malware, misconfigurations, and data leaks.

Think of Kubernetes like a **big apartment building**:

- **Nodes** = buildings
- **Pods/Containers** = apartments
- **API Server** = main gate/control room
- **etcd** = secret locker/database
- **Network** = roads between apartments
- **RBAC** = security guards deciding who can enter

Kubernetes security means protecting:

1. Cluster
2. Nodes
3. Pods
4. Containers
5. Network
6. Secrets
7. API Server
8. Users
9. CI/CD Pipelines
10. Supply Chain

If an attacker enters any one of them, they may compromise the entire cluster.

A simple view:

User

|

API Server

|

etcd (stores cluster data)

|

Nodes

|

Pods

|

Containers

2. Authentication

Who are you?

How kubernetes verify authenticity ,Methods:

- Client Certificates
- Service Accounts
- OIDC
- LDAP
- Cloud IAM
- Keycloak

3. Authorization (RBAC)

What can you do?

Most important Kubernetes security feature.

Bad

verbs: ["*"]

resources: ["*"]

Good

verbs:

- get , -list . -read

Commands for check what you can do in cluster:-

kubectl auth can-i get pods

kubectl auth can-i create pods

kubectl auth can-i delete pods

kubectl auth can-i get secrets

kubectl auth can-i '*' '*' # do anything like admin

* in **verbs** means **all actions** (get, create, update, delete, etc.), and * in **resources** means **all Kubernetes resources** (pods, deployments, services, secrets, nodes, etc.).

4. Service Account Security

Every pod gets a service account token.

Check:

kubectl describe pod <pod>

Disable if not needed:

automountServiceAccountToken: false

5. Secrets Management

Never store passwords in:

env:

- name: PASSWORD

value: admin123

Use Secrets or secret.yaml :-

kubectl create secret generic db-secret \

--from-literal=password=mypass

View:

kubect! get secret

Remember:

Base64 ≠ Encryption

Enable encryption at rest for secrets.

6. Pod Security

Containers should not run as root.

Bad:

securityContext:

runAsUser: 0

Good:

securityContext:

runAsNonRoot: true

Example:

securityContext:

runAsUser: 1000

runAsNonRoot: true

8. Network Policies

Without NetworkPolicy:

Pod A ---> Pod B

Pod A ---> Database

Pod A ---> Everything

With NetworkPolicy:

Pod A ---> Database

Everything else blocked

9. Pod-to-pod attack

By default, many clusters allow pod-to-pod communication.

Example:

frontend pod → backend pod

frontend pod → database pod

attacker pod → database pod

Protection: use NetworkPolicy.

Kubernetes NetworkPolicy controls traffic to and from selected pods using pod selectors, namespace selectors, and IP blocks.

10. Malicious container image

Attack can come from vulnerable or malicious images:

docker pull random/image

Protection:

trivy image nginx

trivy image your-image:tag

Use official/minimal images, scan images, and sign images.

11. HostPath volume attack

Dangerous:

volumes:

- name: host

hostPath:

path: /

This can expose the node filesystem to the pod.

Protection: block hostPath using Pod Security Standards or Kyverno.

12. Service account token abuse

Pods often receive a token automatically.

Attacker inside pod can try:

```
cat /var/run/secrets/kubernetes.io/serviceaccount/token
```

Protection:

automountServiceAccountToken: false

13. API Server attack

The API server is the brain of Kubernetes.

Protect it with:

TLS

Authentication

RBAC

Audit logs

Private endpoint

Admission control

Never expose the API server publicly without strong protection.

14. etcd attack

etcd stores cluster state, secrets, config, tokens.

Protect it with:

TLS

Encryption at rest

Access restriction

Regular backups

15. Runtime security

Even after deployment, monitor suspicious activity.

Attacks:

Shell opened inside container

Crypto miner started

Suspicious network connection

Sensitive file read

Container escape attempt

Tools:

Falco

Tetragon

Tracee

OpenObserve logs

Prometheus/Grafana

Trivy → image and cluster vulnerability scanning

kube-bench → CIS Kubernetes benchmark checks

kube-hunter → cluster penetration testing style checks

Kyverno → policy enforcement

OPA Gatekeeper → policy enforcement

Falco → runtime threat detection

Tetragon → eBPF runtime security

Cosign → image signing

Vault → secret management

What Are the Main Kubernetes Security Issues?

Compromised Images and Image Registries

Compromised Containers or Malicious Traffic

NIX-SERVE Setup

In NixOS, `/nix/store` contains installed packages.

Example:

```
/nix/store/abc123-firefox
/nix/store/xyz456-docker
/nix/store/pqr789-nmap
```

Now imagine you have **3 NixOS machines** in your home/company:

Laptop
PC
Server

When you update all 3 machines, normally each machine downloads packages from internet:

Laptop → downloads Firefox from internet
PC → downloads Firefox from internet again
Server → downloads Firefox from internet again

This wastes internet and time.

Local Nix cache means

You make one machine in your local network act like a **package storage server**.

Example:

NAS / Server = local Nix cache

Then flow becomes:-

Laptop downloads package once



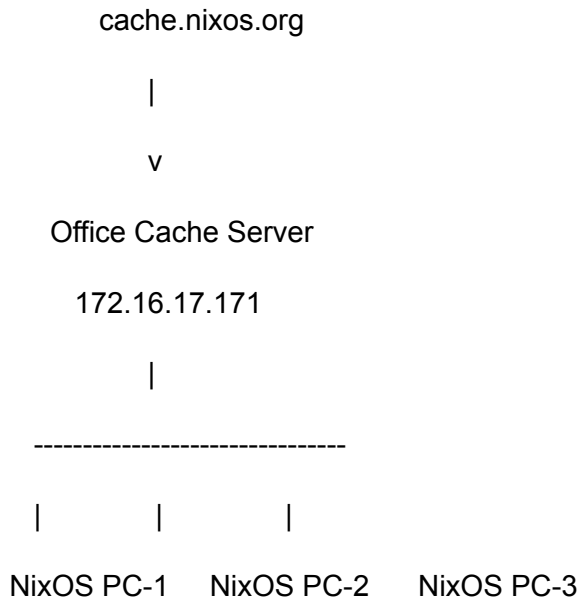
Package is saved in local cache



PC and Server download from local cache instead of internet

So local Nix cache means:

A local network server that stores Nix packages, so other NixOS machines can download from LAN instead of internet.



Step 1: Generate cache keys

Run on your NixOS machine:

```
sudo mkdir -p /var/cache-keys
cd /var/cache-keys
```

```
sudo nix-store --generate-binary-cache-key cache.local-1 cache-private-key.pem
cache-public-key.pem
```

```
sudo chmod 600 cache-private-key.pem
sudo chmod 644 cache-public-key.pem
```

Now show your public key: `cat /var/cache-keys/cache-public-key.pem`

2. Create separate `nix-serve.nix` file

On server PC:

```
sudo nano /etc/nixos/nix-serve.nix
```

Paste this:

```
{ config, pkgs, ... }:
```

```
{
  environment.systemPackages = with pkgs; [
    nix-serve
  ];

  services.nix-serve = {
    enable = true;
    port = 5000;
    secretKeyFile = "/var/cache-keys/cache-private-key.pem";
  };

  networking.firewall.allowedTCPPorts = [ 5000 ];
}
```

3. Import **nix-serve.nix** in main **configuration.nix**

Open:

```
sudo nano /etc/nixos/configuration.nix
```

imports =

```
[
  ./hardware-configuration.nix
  ./nix-serve.nix
];
```

Now rebuild - `sudo nixos-rebuild switch`

Check status:

```
systemctl status nix-serve
```

4. Test server cache locally

Run on **server PC**:

```
curl http://127.0.0.1:5000/nix-cache-info
```

Expected output:

StoreDir: /nix/store

WantMassQuery: 1

Priority: 30

Now test using server IP:

```
curl http://172.16.17.171:5000/nix-cache-info
```

Expected output:

StoreDir: /nix/store

WantMassQuery: 1

Priority: 30

5. Create separate client cache file

On **client PC**:

```
sudo nano /etc/nixos/client.nix
```

Paste this:

```
{ config, pkgs, ... }:
```

```
{
```

```
  nix.settings = {
```

```
    substituters = [
```

```
      "http://172.16.17.171:5000"
```

```
      "https://cache.nixos.org/"
```

```
];
```

```
trusted-public-keys = [
```

```
    "cache.local-1:PASTE-YOUR-PUBLIC-KEY-HERE"
```

```
    "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
```

```
];
```

```
};
```

```
}
```

Now replace this:

```
cache.local-1:PASTE-YOUR-PUBLIC-KEY-HERE
```

6. Import client file in client **configuration.nix**

On **client PC**:

```
sudo nano /etc/nixos/configuration.nix
```

Change it to:

```
imports =
```

```
[
```

```
    ./hardware-configuration.nix
```

```
    ./client.nix
```

```
];
```

7. Rebuild client

Run on **client PC**:

```
sudo nixos-rebuild switch
```

Restart Nix daemon:

```
sudo systemctl restart nix-daemon
```

8. Check client config applied or not

On **client PC**:

```
nix show-config | grep -E "substituters|trusted-public-keys"
```

You should see:

```
substituters = http://172.16.17.171:5000 https://cache.nixos.org/
```

```
trusted-public-keys = cache.local-1:... cache.nixos.org-1:...
```

Easier test using package install

On **server PC**:

```
nix-shell -p cowsay
```

Then exit:

```
exit
```

Now on **client PC**:

```
nix-shell -p cowsay --option substituters "http://172.16.17.171:5000 https://cache.nixos.org/"
```

Then run:

```
cowsay hello
```

What is NIX-SERVE:-

nix-serve is a small server for sharing your Nix store over the network.

In your case:

Your NixOS PC IP = 172.16.17.171

nix-serve port = 5000

So **nix-serve** makes this URL available:

http://172.16.17.171:5000

Your friend's NixOS PC can use this URL as a **local Nix binary cache**.

Simple meaning

`nix-serve` does this:

Your `/nix/store` packages



`nix-serve`



Available on LAN using HTTP



Friend's NixOS PC can download from your machine

binary-cache setup

Server Configuration (**nginx-cache.nix**)

Create `/etc/nixos/nginx-cache.nix`:

```
nix
{ config, pkgs, ... }:

{
  services.nginx = {
    enable = true;
    recommendedGzipSettings = true;
    recommendedProxySettings = true;

    virtualHosts."nix-cache" = {
      listen = [ { addr = "0.0.0.0"; port = 5000; } ];
      locations."/" = {
        proxyPass = "https://cache.nixos.org";
        extraConfig = "
          proxy_set_header Host cache.nixos.org;
          proxy_cache nixcache;
          proxy_cache_valid 200 302 60d;
        ";
      };
    };

    appendHttpConfig = "
      proxy_cache_path /var/cache/nginx/nix levels=1:2 keys_zone=nixcache:100m
      max_size=10g inactive=60d use_temp_path=off;
    ";
  };

  networking.firewall.allowedTCPPorts = [ 5000 ];

  # Add your public key so clients can trust your cache
  nix.settings.trusted-public-keys = [
    "cache.local-1:M1QOVzJ5XZ+sl8tENaDnScx4HCd6OFzZC6D8Qg0iwK4="
  ];
}
```

Then import it in `/etc/nixos/configuration.nix`:

```
nix
```

```
imports = [
  ./hardware-configuration.nix
  ./nginx-cache.nix
];
```

Rebuild:

```
bash
sudo nixos-rebuild switch
```

Client Configuration

On each client machine, add:

```
nix
nix.settings = {
  substituters = [
    "http://<server-ip>:5000"
    "https://cache.nixos.org/"
  ];
  trusted-public-keys = [
    "cache.local-1:M1QOVzJ5XZ+sl8tENaDnScx4HCd6OFzZC6D8Qg0iwK4="
    "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
  ];
};
```

Replace `<server-ip>` with your server's LAN IP (e.g. `172.16.17.171`).

How It Works

- **Client1 installs `nmap`** → fetched from `cache.nixos.org`, stored in `/var/cache/nginx/nix`.
- **Client2 installs `nmap`** → served directly from your server's cache, no internet needed.
- **Server itself installs `nmap`** → it will also fetch from its own proxy cache instead of the internet.

nixos-paste-config-issue

Step 1: Boot into root shell

At NixOS boot menu:

1. Select your NixOS generation
2. Press **e**
3. Find the line that starts with:

linux

4. At the end of that line, add:

init=/bin/sh

5. Press **Ctrl + X** or **F10** to boot.

Now you will get root shell.

Step 2: Remount system as writable

Run:

```
mount -o remount,rw /
```

Step 3: Open your NixOS config

Run:

```
nano /etc/nixos/configuration.nix
```

Now check your user section.

You should have something like this:

```
users.users.priyanshu = {  
  isNormalUser = true;  
  description = "Priyanshu";  
  extraGroups = [ "networkmanager" "wheel" ];  
};
```

Make sure username is your real username:

priyanshu

Do **not** keep someone else's username.

Step 4: Remove wrong password settings

Inside `configuration.nix`, search for lines like these:

```
initialPassword = "...";  
hashedPassword = "...";  
hashedPasswordFile = "...";
```

Remove them if they are from someone else.

Example clean user config:

```
users.users.priyanshu = {  
  isNormalUser = true;  
  description = "Priyanshu";  
  extraGroups = [ "networkmanager" "wheel" ];  
};
```

Save file:

Ctrl + O

Enter

Ctrl + X

Step 5: Rebuild system

Run:

`nixos-rebuild switch`

If this command gives PATH error, run this instead:

`/run/current-system/sw/bin/nixos-rebuild switch`

Step 6: Set new password

Now set your password again:

```
passwd priyanshu
```

Enter new password two times.

Also set root password if needed:

```
passwd root
```

Step 7: Reboot

Run:

```
reboot -f
```

Now login with your username:

```
priyanshu
```

and the new password you just created.

Important

After login, again open config:

```
sudo nano /etc/nixos/configuration.nix
```

Keep your user section like this:

```
users.users.priyanshu = {  
  isNormalUser = true;  
  description = "Priyanshu";  
  extraGroups = [ "networkmanager" "wheel" ];  
};
```

Then rebuild:

```
sudo nixos-rebuild switch
```

owasp top10 k8s

Top 10 Kubernetes Risks - 2025

- K01: Insecure Workload Configurations
- K02: Overly Permissive Authorization Configurations
- K03: Secrets Management Failures
- K04: Lack Of Cluster Level Policy Enforcement
- K05: Missing Network Segmentation Controls
- K06: Overly Exposed Kubernetes Components
- K07: Misconfigured And Vulnerable Cluster Components
- K08: Cluster To Cloud Lateral Movement
- K09: Broken Authentication Mechanisms
- K10: Inadequate Logging And Monitoring

K01: Insecure Workload Configurations - Unsafe Pod Settings

What is it?

Your Pods/Containers are running with dangerous settings.

Example:

securityContext:

privileged: true

runAsUser: 0 # run as 0 means running as root.

How attackers attack?

Suppose an attacker finds an RCE (Remote Code Execution) vulnerability in your application.

Attacker perform command injection on RCE

If the container is:

- Running as root
- Privileged : true

How to secure?

securityContext:

```
runAsNonRoot: true
allowPrivilegeEscalation: false
readOnlyRootFilesystem: true
```

capabilities:

```
drop:
- ALL
```

Linux capabilities drop all # its a linux capabilities that tells to container runasnonroot : true it doesnt give any admin permission.

K02: Overly Permissive Authorization Configurations - Too many Permissions

Example of a Bad Configuration

rules:

```
- apiGroups: ["*"] # * means you have admin cluster
```

```
resources: ["*"]
```

```
verbs: ["*"]
```

Correct Way For RBAC is :-

rules:

```
- apiGroups: [""] # "" indicates the core API group
```

```
resources: ["pods", "pods/log"] # Explicitly specify resources
```

```
verbs: ["get", "list", "watch", "update"] # Explicitly specify allowed actions
```

```
- apiGroups: ["apps"] # Non-core API group
```

```
resources: ["deployments"]
```

```
verbs: ["get", "list", "update", "patch"]
```

K03: Secrets Management Failures - Poor Secret Protection

Never Store Secrets in env file and config file always stores in secret.yaml

Common K03 Mistakes

Mistake 1: Secrets in Git

Mistake 2: Secrets in ConfigMaps

Mistake 3: Secrets in Environment Variables

Mistake 4: Kubernetes Secrets Are Not Encrypted